Data Structures and Algorithms

Instructor: Dr. Hao Qin（秦浩）

E-mail: hao.qin@scupi.cn

My Office: Room 514, SCUPI (North)

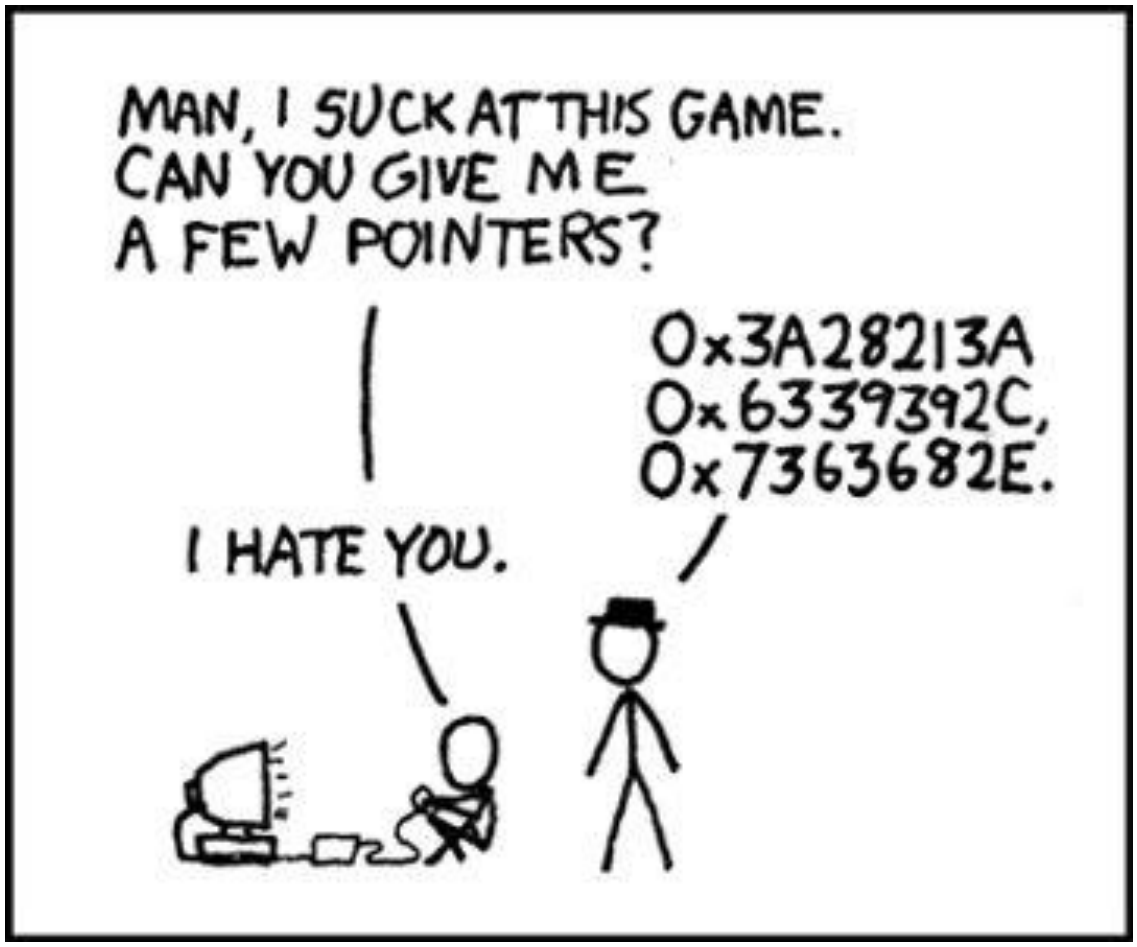
Lecture #3

- Pointers:
  - A Quick Review of Pointers
  - Dynamic Memory Allocation
- Resource Management Part 1:
  - Copy Constructors

If you feel uncomfortable with pointers, then study and become an expert before our next class!

(Yeah right… like you're gonna review on your own)

Pointers

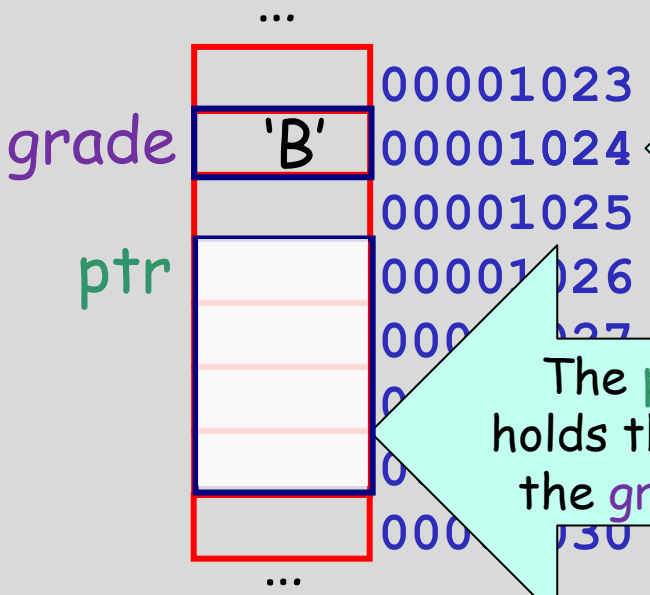

MAN, I SUCK AT THIS GAME.
CAN YOU GIVE ME
A FEW POINTERS?
0x3A28213A
0x6339392C,
0x7363682E.
I HATE YOU.

Addresses and Pointers…
What's the big picture?

We use pointers to efficiently access/modify variables defined in other parts of our program.

Just like every house has a street address, every variable has a memory address.

```
void someFunction()
{
    char grade = 'B';
    …

    char *ptr = &grade;
}
```



The grade variable has an address of 1024 in RAM

The ptr holds the the grad

Uses:

Pointers are used in all C++ programs to efficiently pass parameters, and to refer to dynamic variables.

A pointer is simply a variable that holds another variable's address!

Every Variable Has An Address

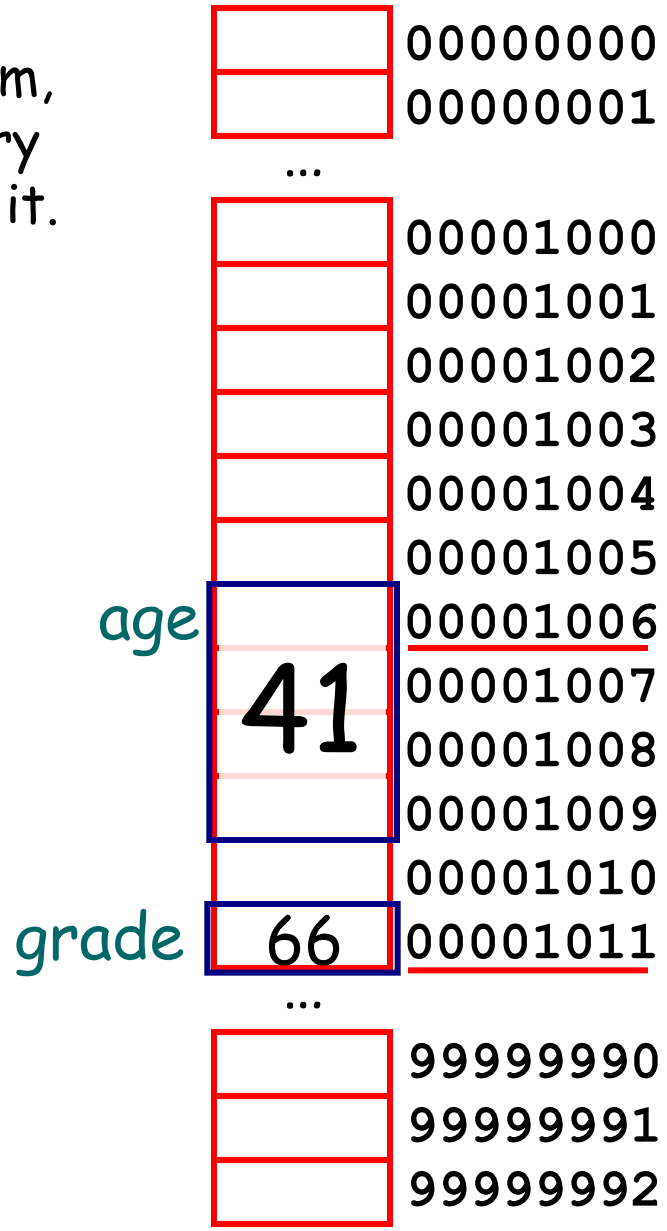
Every time you define a variable in your program, the compiler finds an unused address in memory and reserves one or more bytes there to store it.

Important: The address of a variable is defined to be the *lowest* address in memory where the variable is stored.

So, what is age's address in memory?

What about grade's address?

```
int main()
{
    int   age = 41;
    char grade = 'B';

    ...

}
```



Getting the Address of a Variable

We can get the address of a variable using C++'s & operator.

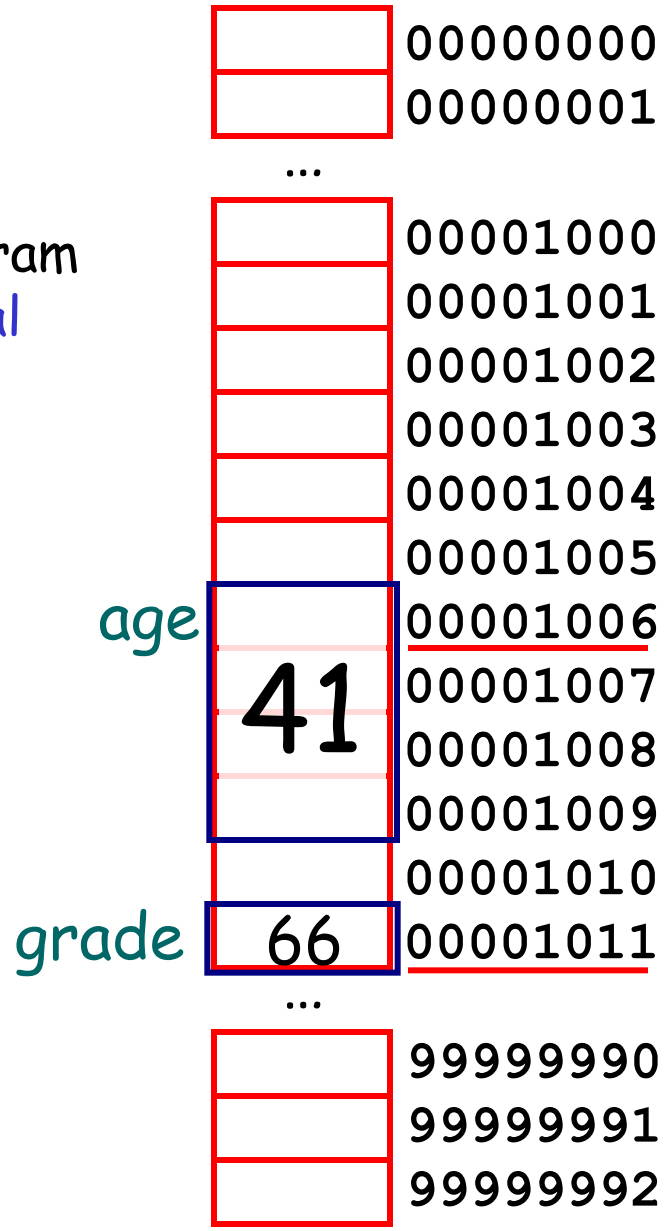
If you place an & before a variable in a program statement, it means "give me the numerical address of the variable."

Output:

age's address: 1006

grade's address: 1011

```
int main()
{
   int    age = 41;
   char grade = 'B';

→ cout << "age's address: "<< &age ;
→ cout << "grade's address: " << &grade ;

}
```



Ok, So What's a Pointer?

pointer variable

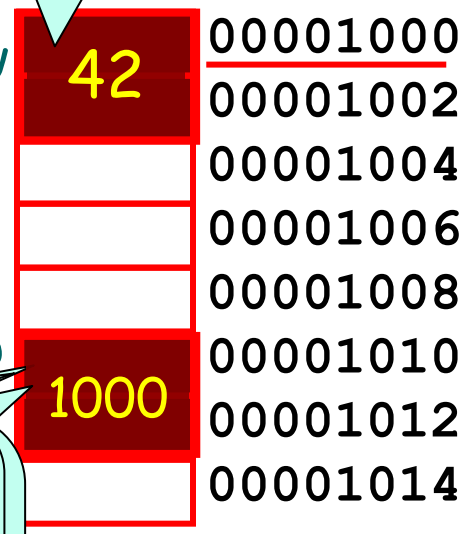
A ~~pointer~~ is a special kind of variable that holds another variable's address instead of a regular value.

I'm a regular variable… and I hold a regular value!

Here's how we define a regular variable!

"Variable p is a pointer to an int variable."

```
void foo()
{
  int idontknow;
  idontknow = 42;
  int *p;
  p = &idontknow;
}
```



the address of the "idontknow" variable.

To understand the type of your pointer variable, simply read your declaration from right to left…

What do I do with Pointers?

Question: So I have a pointer variable that points to another variable… now what?

Answer: You can use your pointer and the star operator to [illegible] the other variable.

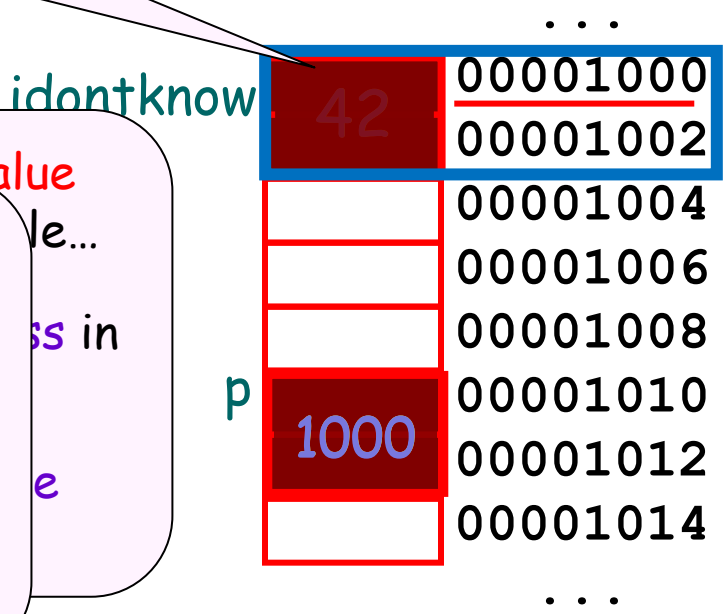
I just changed idontknow's value to 5…

```
void foo()
{
    int idontk
    idontknow

    int *p;
    p = &id

→ cout <<          ;
→ *1000= 5;
}
```

"Get the address value stored in the p variable…

Then go to that address in [illegible]

…without ever referring to its variable name!



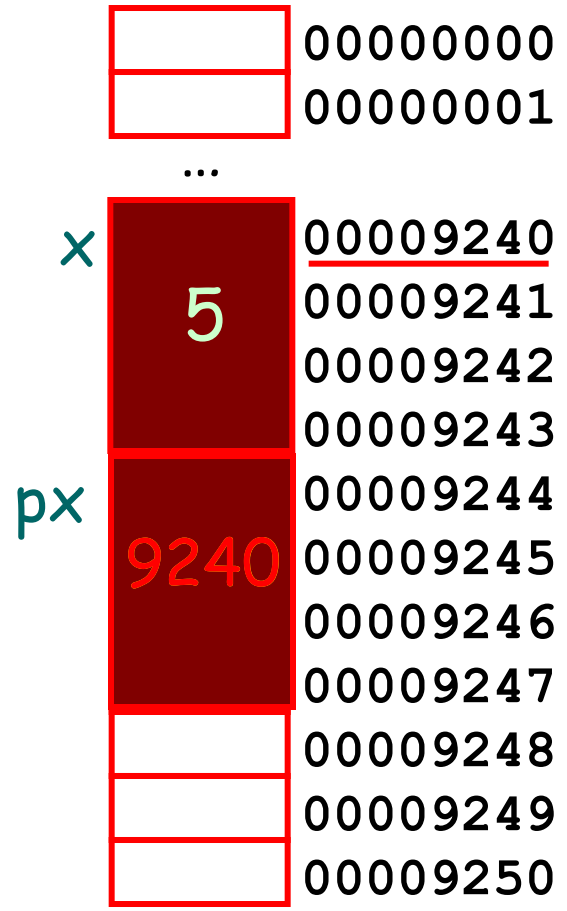
cout << *p → cout << *1000 → cout << 42

*p = 5 → *1000 = 5

Another Pointer Example

```
void set(int *px)
{
  *px = 5;
}

int main()
{
  int x = 1;
      9240
  set(&x);

  cout << x; // prints 5
}
```

"Store a value of 5 at location ."



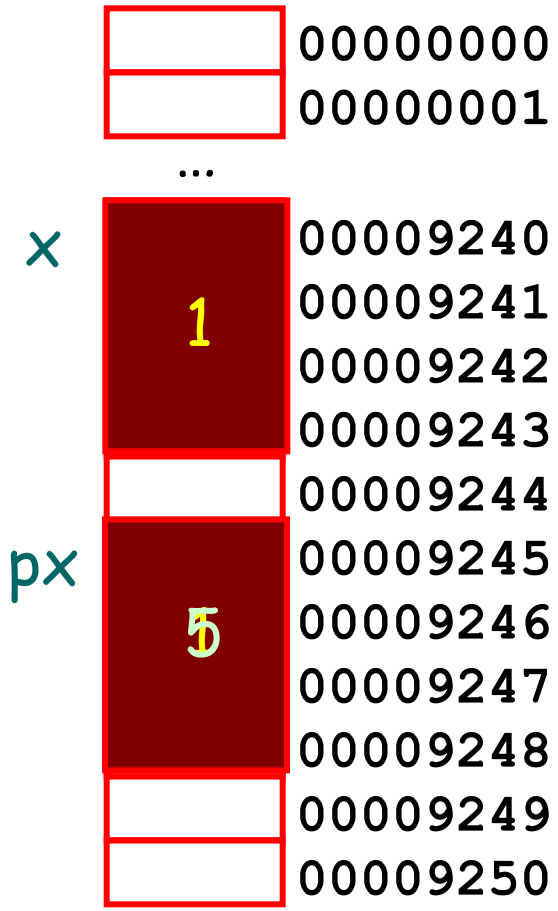
Let's use pointers to modify a variable inside of another function.

Cool – that works! We can use pointers to modify variables from other functions!

What if We Didn't Use Pointers?

```
void set(int  px)
{
  px = 5;
}

int main()
{
 int x = 1;
        1
 set( x);

 cout << x; // prints 1
}
```

Now what would happen if we didn't use pointers in our code?



Oh no! We tried to change the value of x in set but it only changed the local variable!

Had we used a pointer, it would have worked!

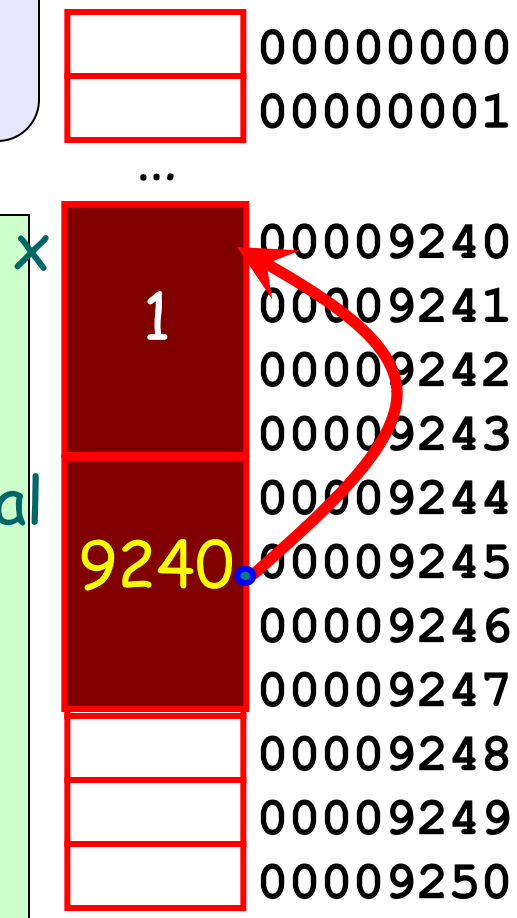
Pointers vs References

When ... to a ... ction ... ce...

Since val points to our original variable, x, this line actually changes x!

```
void set(in      ) // val is a ref
{
  val =  5
}

int mai
{
  int x =
          9240
  set(x);
  cout << x;
}
```

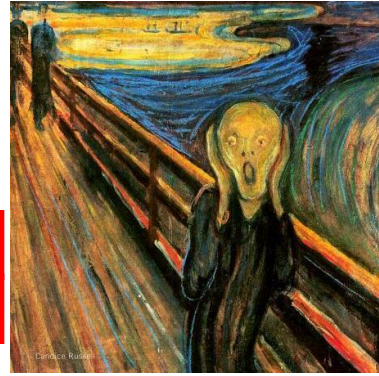
This line is really passing the address of variable x to set...



In fact, a reference is just a simpler notation for passing by a pointer!

(Under the hood, C++ uses a pointer)

Pointers are Dangerous!



You must alw
a pointer v

Or it could point at a critical variable!

(But not the one you want!)

So what address does our pointer hold?

It's random!!

```
int main()
{
   double bank
   double colle

   ...

   double *ptr_to_debt  = nullptr;
   ptr_to_debt = &college_debt;
   *ptr_to_debt = 0; /
}
```

bank_balance

college_debt

hat would happen
if we accidental
deleted or just fo
this line?

	00009240
300.50	00009244
	00009248
	00009252
5000	00009256
	00009260
	00009264
[illegible]	00009268
	00009272
	00009276
	00009280

CRASH!

Pro tip:
imm

ight point at an
npty slot of memory...

our debt to

Why? If you use * on a null pointer, your program will crash immediately and you'll find the bug ASAP!

Arrays, Addresses and Pointers

Just like any
array has ar

But… in C+
& operato

You simply wr
(without brackets) and C++ will
give you the array's address!

re's how to make a pointer
point to an array…

o is "nums" an address or a
pointer or what?

"nums" is just an array.
u get its address without
& it looks like a pointer…

nums is just an array.

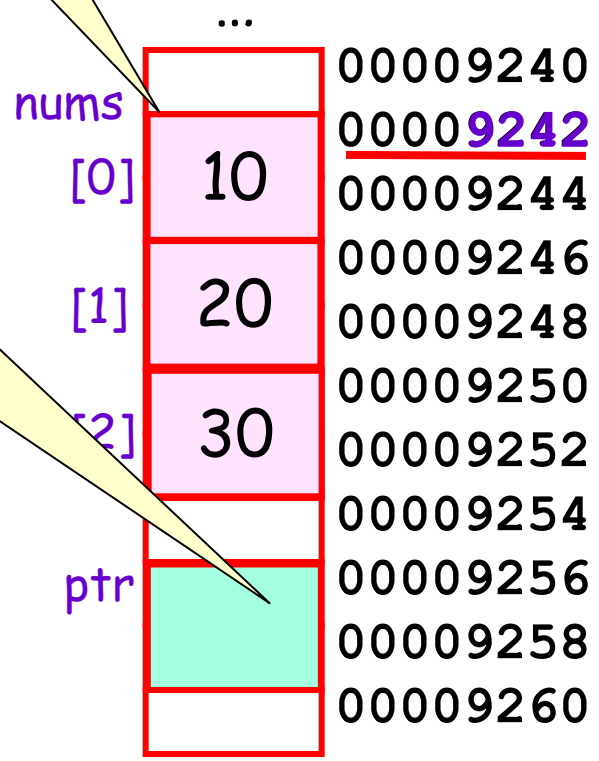
It holds three regular integer values.

But it doesn't hold an address like a pointer variable, so it's not a pointer!

ptr is a pointer variable.

Why? Because it's a variable that holds an address value!

```
int main()
{
int nums[3] = {10,

cout <<          ; // prints 9242

int *ptr = nums;   // pointer to array


}
```



Arrays, Addresses and Pointers

In C++, the two syntaxes have identical behavior:

ptr[j] ↔ *(ptr + j)

"Get the value in ptr, and go to that address in memory, then skip down j elements and get the value."

NOTE: when we say "skip down j elements," we don't just mean "skip down j bytes!"

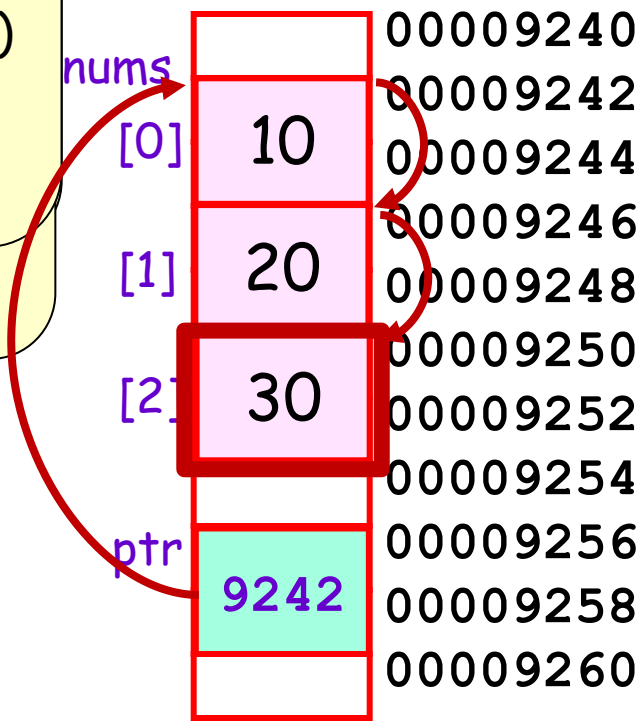
Instead we mean, skip over j of the actual elements/values in the array (e.g., skip over the values 10 and 20 to get to 30)

This line says "print out the item that is two elements down from where ptr points." (in this case, ptr points to the top of the array)

It's the same as printing out nums[2] or ptr[2].

...e nums array.

```
int main()
{
   int nums[3] = {
   int *ptr = nums;

   cout << ptr[2];  // prints nums[2] or 30
   cout << *ptr;    // prints nums[0] or 10
   cout << *(ptr+2);// prints nums[2] or 30

}
```



P… and Arrays

The array parameter variable is actually a pointer!

You can use [ ] syntax if you like but it's REALLY a pointer!

…prints the element that …e item from the top.

Since array starts at 3000, and each item is an integer, our next integer is at 3004…

…u pass …

… the …e array!

… not the array itself!

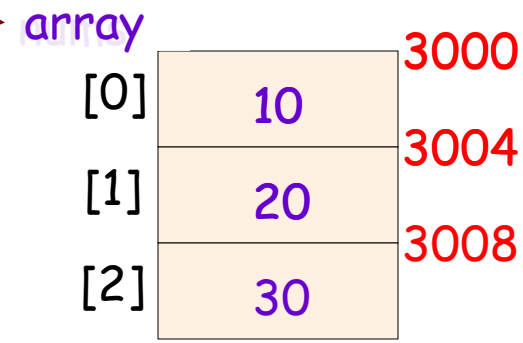
array

```
void printData(int *array)
{
   cout << array[0] << "\n";
   cout << arra…
}
```

```
int main()
{
   int nums[3] …

   printData(nu…

   printData(&nums[1]);

   printData(nums+1);
}
```

Here we're passing the address of the second element of the array.

Since nums[0] is at address 3000, nums[1] is 4 bytes down at 3004.



Pointer

Our printData function thinks the array actually starts at location 3004!

It kn

This prints the element that's zero items

and this prints the item that is one item down from the start of the array…

When you use recursion on arrays, you'll often use this notation…

This line is tricky!

First, what happens when you just write

Then when we add 1, we tell C++ we want the address of the element that is one down from the top of the array.

array 3004

```
void printData(int array[])
{
   cout << array[0] << "\n";
   cout << array[1] << "\n";
}
```

```
int main()
{
   int nums[3] = {10,20,30};

   printData(nums);

   printData(&nums[1]);

   printData(nums+1);
}
```

3004 one array element down

10
20

nums + 1 points here!

nums

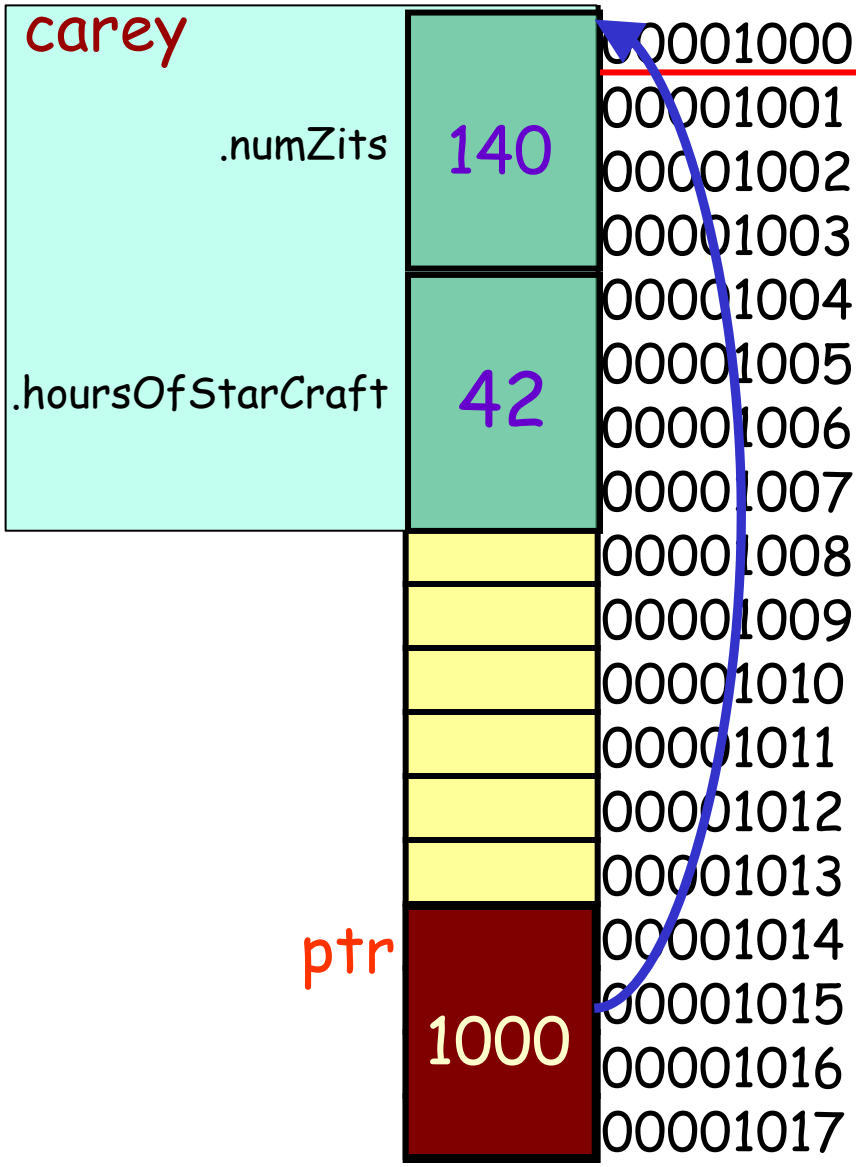
	10	3000
	20	3004
[1]	30	3008

Pointers Work with Structures Too!

You can use pointers to access structs too! Use the * to get to the structure, and the dot to access its fields.

Or you can use C++'s -> operator to access fields!

```
struct Nerd
{
  int numZits;
  int hoursOfStarCraft;
};

int main()
{
  Nerd  carey;
  Nerd  *ptr;

  ptr = &carey;

  (*ptr).numZits = 140;
  ptr->hoursOfStarCraft = 42;
}
```

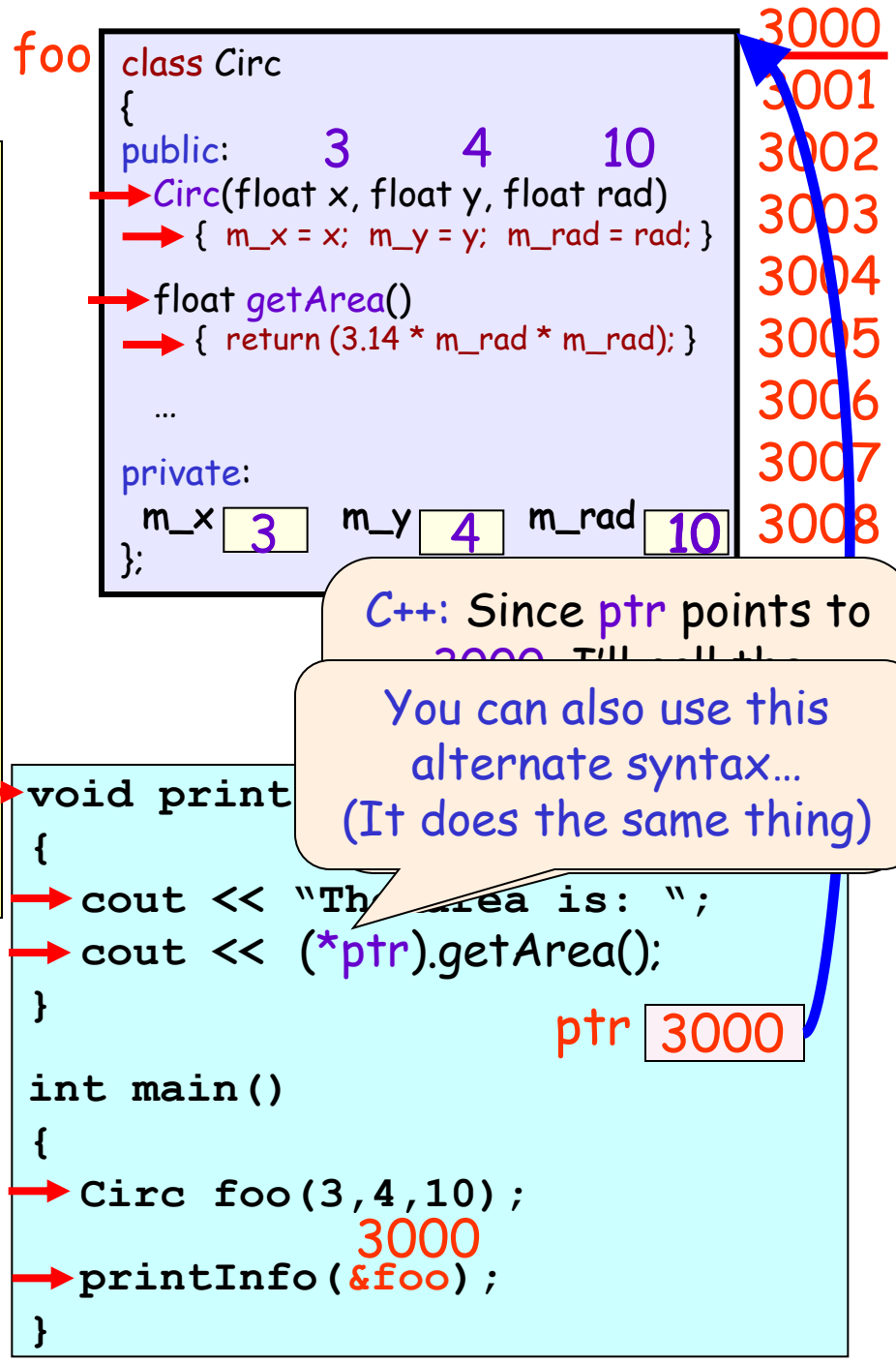


Classes and Pointers

```
class Circ
{
public:
   Circ(float x, float y, float rad)
      { m_x = x;  m_y = y;  m_rad = rad; }

   float getArea()
      { return (3.14 * m_rad * m_rad); }

    ...

private:
  float m_x, m_y, m_rad;
};
```

You can use pointers with classes just like you do with structs.

The area is: 314



The Old Days Before Classes

A Wallet structure keeps track of how many $1 and $5 the

The function use its poi..er the original v

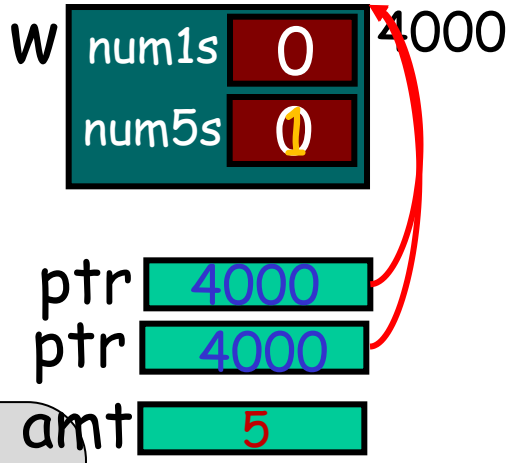
The Init() function initializes the wallet...

It's like a constructor.

And the AddBill() function lets us add a bill to the wallet...

by updating the contents of the struct.

...tion ...ruct ...ed to ...n the variable's address...




```
stru
{
   in
};

void  it(Wallet
{
→ ptr->num1s = 0
→ ptr->num5s =
}

→void AddBill(Wallet *ptr, i
{
→ if (amt == 1) ptr->num1s++;
→ else if (amt == 5) ptr->num5s++;
}
```

```
ain()

 llet w;
 000
→Init(&w);
        4000  5
→AddBill(&w , 5);
}
```

As it turns out, C++ classes work in an almost identical fashion!

The Wallet Class

```
class Wallet
{
public:
   void Init();
   void AddBill(int amt);
   …
private:
   int num1s, num5s;
};

void Wallet::Init()
{
    num1s =   num5s = 0;
}
void Wallet::AddBill(int amt)
{
   if (amt == 1)      num1s++;
   else if (amt == 5)num5s++;
}
```

Here's a class equivalent of our old-skool Wallet…

As you can see, we can initialize a new wallet…

And we can add either a $1 or $5 bill to our wallet.

Our wallet then keeps track of how many bills of each type it holds…

And here's how we might use our class…

```
int main()
{
  Wallet a;

  a.Init();
  a.AddBill(5);
}
```

Classes and the "this" Pointer

And C ... our actual r...

It adds a hidd... argument that's a pointer to y... ginal variable!

Here what your Init() method looks like...

But here's what's REALLY happening! ☺

```
void Wallet::Init()
{
    num1s =   num5s = 0;
}
void Wallet::AddBill(int amt)
{
   if (amt == 1)      num1s++;
   else if (amt == 5)num5s++;
}
...
```

```
void WalletInit(Wallet  *this)
{
   this->num1s =  this-> num5s = 0;
}
void  Wallet::AddBill(this, amt)
{
  if (amt == 1)       this-> num1s++;
  else if (amt == 5) this-> num5s++;
}
...
```

So here we're calling the Init() method of a...

But here's what's REALLY happening! ☺

```
int
{
  Wallet a, b;

  a.Init();
  b.AddBill(5);
}
```

```
int ma
{
  Wallet , b;

  a.Init(&;
  b.AddBill(&5),;
}
```

Classes and the "this" Pointer

C++ converts all of your member functions automatically and invisibly by adding an extra pointer parameter called "this":

Yes… the pointer is actually called "this"!

```
void Wallet::Init()
{
    num1s =   num5s = 0;
}
void Wallet::AddBill(int amt)
{
   if (amt == 1)      num1s++;
   else if (amt == 5)num5s++;
}
...
```

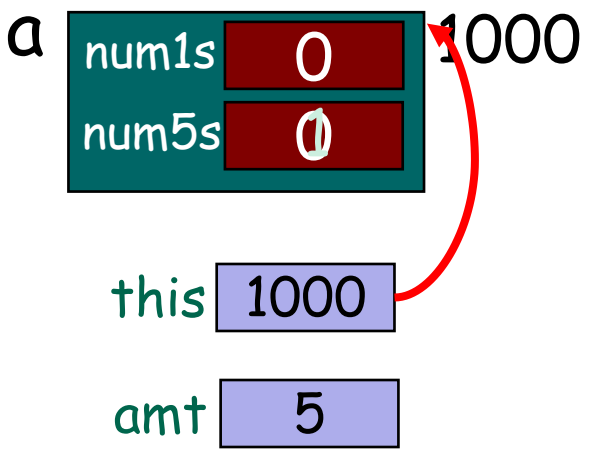
```
void Init(Wallet *this)
{
  this->num1s = this->num5s = 0;
}
void AddBill(Wallet *this, int amt)
{
   if (amt == 1)     this->num1s++;
   else if (amt==5)  this->num5s++;
}
...
```

```
int main()
{
  Wallet a, b;

  a.Init();
  a.AddBill(5);
}
```

```
int main()
{
  Wallet a, b;

  Init(&a);
  AddBill(&b,5);
}
```

Classes and the "this" Pointer

```
void Wallet::Init(              )
{
        num1s =         num5s = 0;
}
void Wallet::AddBill(                  int amt)
{
 if (amt == 1)          num1s++;
 else if (amt==5)        num5s++;
}
...
```




```
int main()
{
 Wallet a;
          1000
 a.Init(  );
               1000  5
 a.AddBill(      5);
}
```

This is how it actually works under the hood….

But C++ hides the "this pointer" from you to simplify things.

Classes and the "this" Pointer

You can explicitly use the "this" variable in your methods if you like!

It works fine!

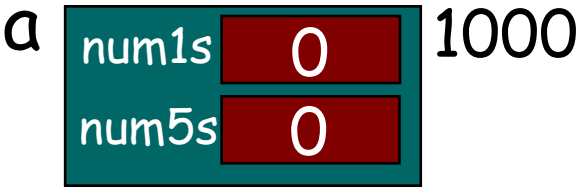
pointer" from you, if you
ds can explicitly use it.

```
void Wal      .:Init()
{
  this->num1s = this->num5s = 0;
  cout << "I am at address: " << this;
}
void Wallet::AddBill(int amt)
{
   if (amt == 1)                num1s++;
   else if (amt == 5)           num5s++;
}
...
```

```
int main()
{
Wallet a;

a.Init();
cout << "a is at address: " << &a;
}
```

Your class's methods can use the this variable to determine their address in memory!

So now you know how C++ classes work under the hood!



I am at address: 1000
a is at address: 1000

unctions?!?

Just like every variable, every function has an address in memory too!

```
3000 void squared(int a)
3050 { cout << a*a;}
3100
3150 void cubed(int a)
3200 { cout << a*a*a;}
3250
3300
3350
3400
3450
3500
3550 int main()
3600 {
3650 FuncPtr f;
3700
3750 f = &squared;
3800 f(10);
3850 f = &cubed;
3900 f(2);
3950 }
```

100

8

10

3000

3150

f

YES! Just as you can have pointers to variables, in C++ you can also have pointers to functions!

Let's gloss over the syntax for a second, and just see how it might work…

Now we can use the function pointer just like a regular function call!

dress
ction
f.

Pointers… to Functions?!?

```
3000 void squared(int a)
3050   { cout << a*a;}
3100
3150 void cubed(int a)
3200   { cout << a*a*a;}
3250
3300
3350
3400
3450
3500
3550 int main()
3600 {
3650     void (*f)(int);
3700
3750     f = &squared;
3800     f(10);
3850     f = &cubed;
3900     f(2);
3950 }
```

YES! Just as you can have pointers to variables, in C++ you can also have pointers to functions!

Let's gloss over the syntax for a second, and just see how it might work…

And here's the real syntax to declare a function pointer…

Oh, and you can leave out the & if you like.

It works the same way!

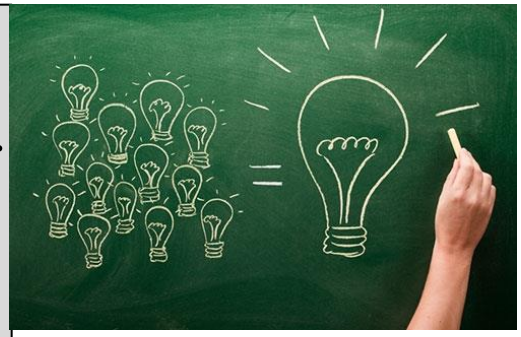
…out the … now…

…er the concept.

Dynamic Memory Allocation…
What's the big picture?

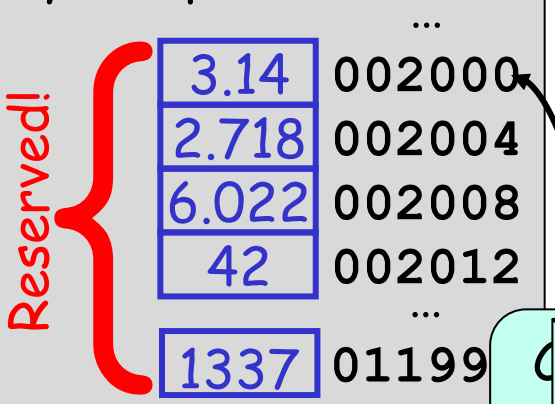
Often a program won't know how much memory it needs to solve a problem until it's actually running.

In these cases, C++ lets you reserve a chunk of memory on-demand, and gives you a pointer to it.



Your code can use the memory via the pointer.

When you're done, you then ask C++ to unreserve it!




```
void computeSomethingImportant(int count)
{
→ ptr = askC++ForThisMuchMemory(count);
→ operateOnTheMemoryAt(ptr);
→ tellC++WereDoneWithThisMemory(ptr);
}
```

ptr

200

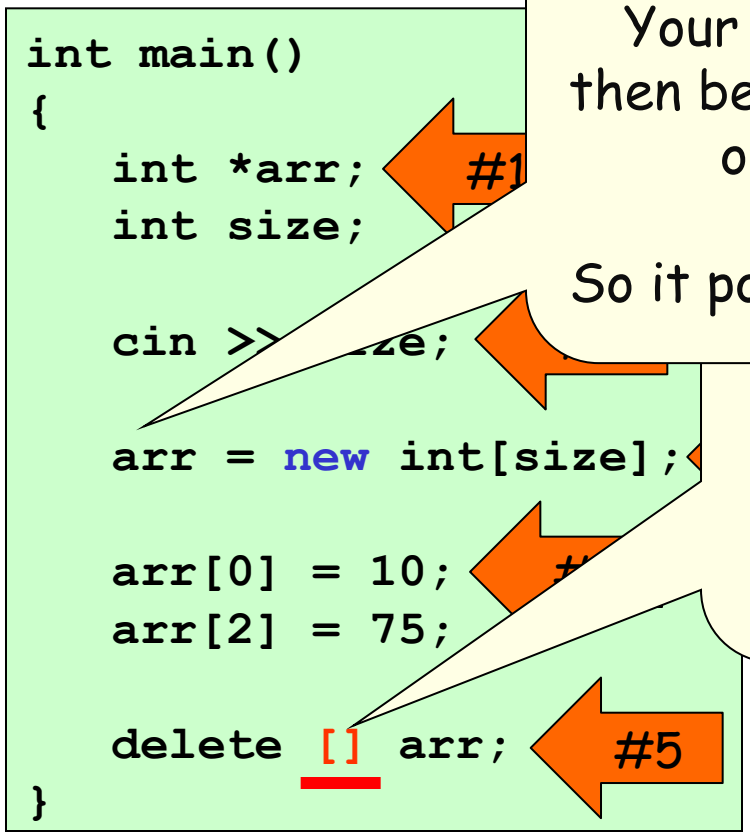
"Hey th

Uses:

Dynamic allocation is used in video games, word processors, search engines, etc., etc.

New and Delete (For Arrays)

Let's say we want to define an array, but we won't know how big to make it until our program actually runs …



Your pointer variable will then be assigned the address of the new array.

So it points to the new array!

delete [ ] ptr;

if you're deleting an array…

…be used to allocate an …emory for an array.

…pointer variable.

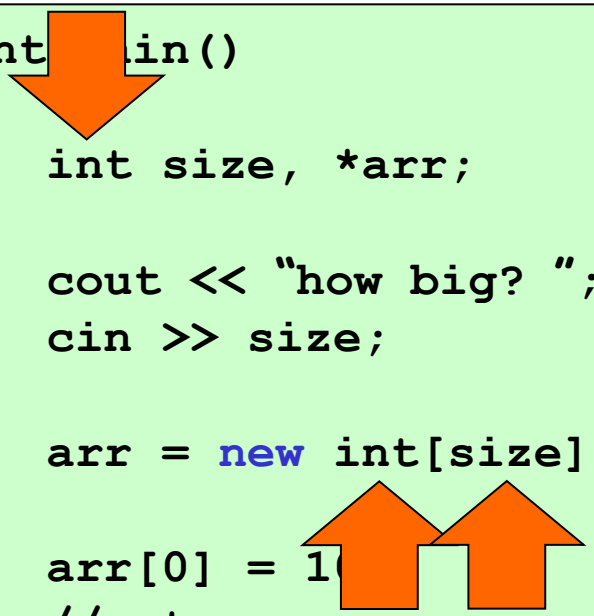
…size of the …

…ommand to …y. Your pointer …f the memory.

4. Now use the pointer just like it's an array!
5. Free the memory when you're done.

New and Delete (For Arrays)

```
int main()
{
    int size, *arr;

    cout << "how big? ";
    cin >> size;

    arr = new int[size];

    arr[0] = 1;
    // etc

    delete [] arr;
}
```



The new command requires two pieces of information:

1. What type of array you want to allocate.
2. How many slots you want in your array.

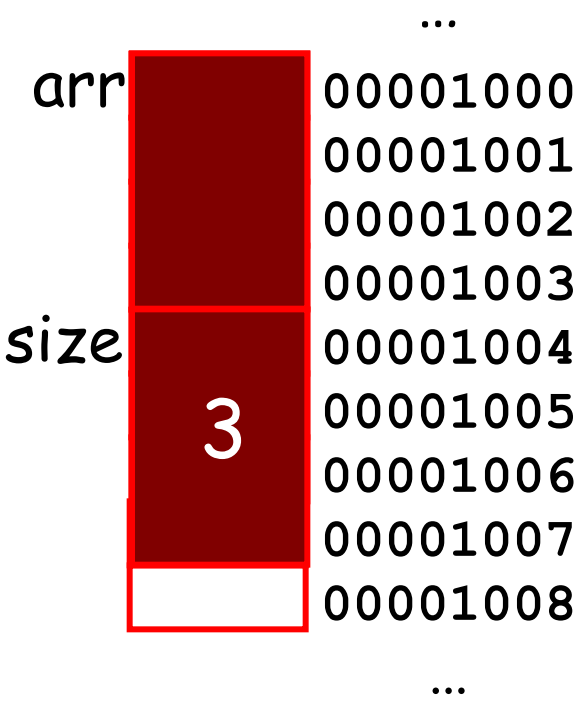
Make sure that the pointer's type is the same as the type of array you're creating!

New and Delete (For Arrays)

```
int main()
{
   int *arr;
   int size;

   cin >> size;

   arr = new int[size];

   arr[0] = 10;
   // etc

   delete [] arr;
}
```

3

4 * 3 = 12 bytes



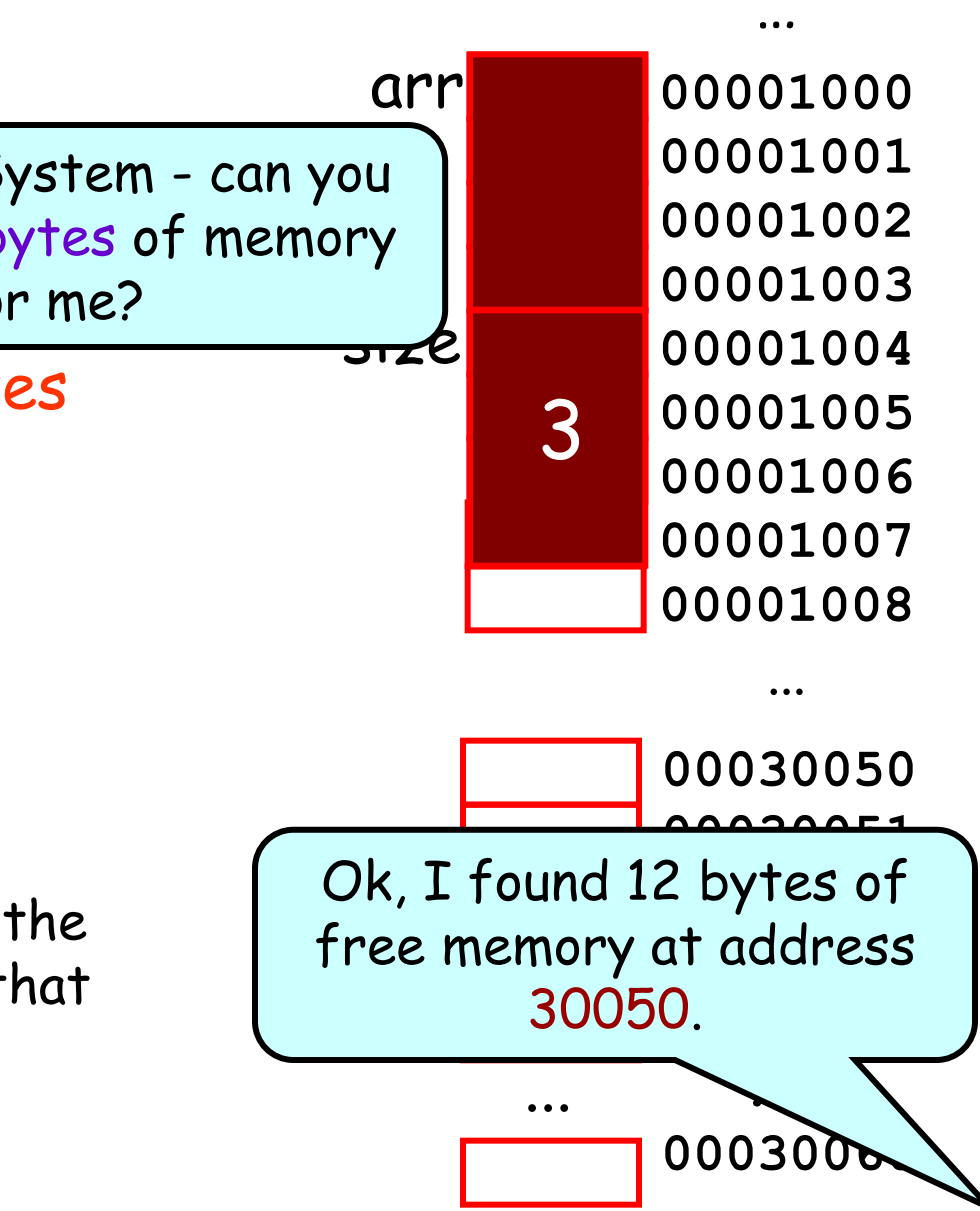
First, the new command determines how much memory it needs for the array.

New and Delete (For Arrays)

```
int main()
{
    int *arr;
    int size;

    cin >> size

    arr = new int[size];

    arr[0] = 10;
    // etc

    delete [] arr;
}
```

Operating System - can you reserve 12 bytes of memory for me?

4 * 3 = 12 bytes

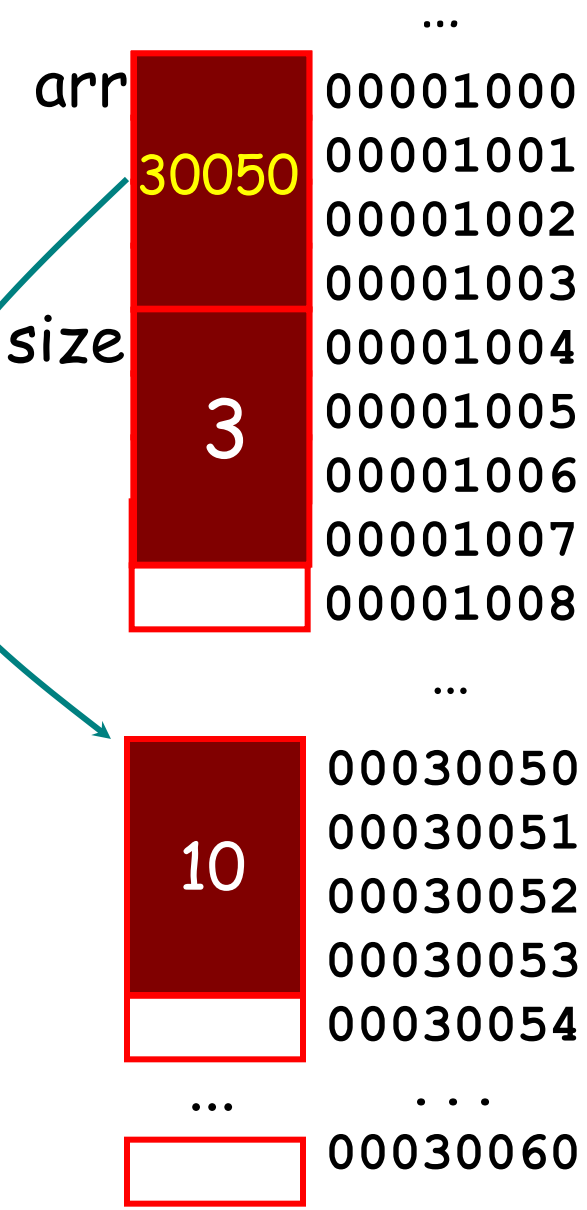


Next, the new command asks the operating system to reserve that many bytes of memory.

New and Delete (For Arrays)

```
int main()
{
    int *arr;
    int size

    cin >> s

    arr = new
    *(arr+0) = 10;  // arr[0] = 10;
    *(arr+1) = 20;  // arr[1] = 20;

    delete [] arr;
}
```

You can now treat your pointer just like an array! (i.e. use [ ] to index it)



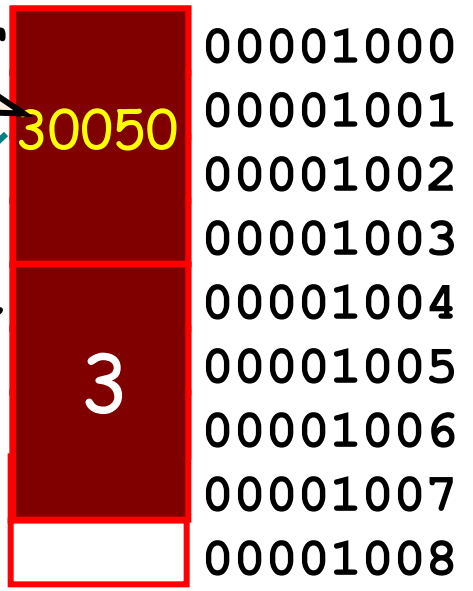
Finally, your pointer variable gets the address of the newly reserved memory.

New … r Arrays)

```
int main()
{
    int *arr
    int size;

    cin >> size;

    arr = new

    arr[0] =
    // etc

    delete [] arr
    arr[0] = 50;
}
```

… not the pointer variable itself!

Our pointer variable still holds the address of the previously-reserved memory slots!



Operating System – I'm done with my 12 bytes of memory at location 30050.

command, you ee the pointed- to memory…

CRASH!

When yo… delete comma…

Usage: de…

But they're no longer reserved for this program!

So don't try to access them or bad things will happen!

k.. I'll let someone else se that memory now…

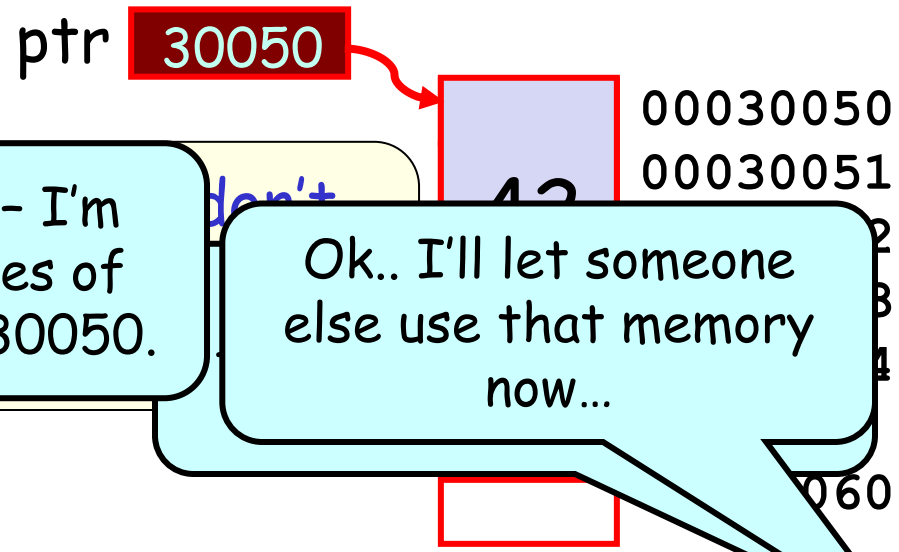
New and Delete (For Non-Arrays)

We can also use new and delete to dynamically create other types of variables as well!

```
int main()
{
    // define our po
    int *ptr;

    // allocate our dynamic variable
    ptr = new int;

    // use our dyn
    *ptr = 42;
    cout << *ptr <<

    // free our dynamic variable
    delete ptr;
}
```

, we can allocate
ariable like this…

Operating System – can you reserve 4 bytes of memory for me?



Operating System – I'm done with my 4 bytes of memory at location 30050.

Ok.. I'll let someone else use that memory now…

New and Delete (

We can also use new and
create other types

```
struct Point
{
   int x;
   int y;
};
```

```
int main()
{
    // define our po
    Point *ptr;

    // allocate our dynamic variable
    ptr = new Point;

    // use our dy
    ptr->x = 10;
    (*ptr).y = 20;

    // free our dyn      variable
    delete ptr;
}
```

Operating System – can you reserve 8 bytes of memory for me?

Operating System – I'm done with my 8 bytes of memory at location 30050.

Ok.. I'll let someone else use that memory now…

For instance, we can allocate
er variable like this…

can allocate a struct
variable like this….

New and Delete (

We can also use new and
create other types

```
class Nerd
{
  public:
    Nerd(int IQ, int zits)
    {
      m_myIQ = IQ;
      m_myZits = zits;
    }
    void saySomethingNerdy()
    {
      cout << "C++ rocks!";
```

```
int main()
{
    // define our pointer
    Nerd *ptr;

    // allocate our dynamic variable
    ptr = new Nerd(150, 1000);

    // use our dy    ic variable
    ptr->saySomet    erdy();

    // free ou
    delete ptr;
    delete ptr;
}
```

Then calls the Nerd constructor with these parameters to initialize it!

class instance like this....

This allocates enough memory for a Nerd variable...

Using new

```
class PiNerd
{
public:
    PiNerd(int n) {
      m_pi = new in
      m_n = n;
      for (int j
        m_pi[i]
   }

   ~PiNer
      dele
   }

   void showOf
   {
      for (int
        cout <
    }
private:
                    int *m_pi, m_n;
};
```

Step #3:

Let's assume we're given a function that can give us any digit of π.

Step #

Add a destr or that frees the dynamic array when we're done!

Cool! Now we can have PiNerds of varying nerdiness!

#5:

our loop so we print numbers.

Change our fixed ay to a pointer variable and add a size variable.

compute nbers irst 10)

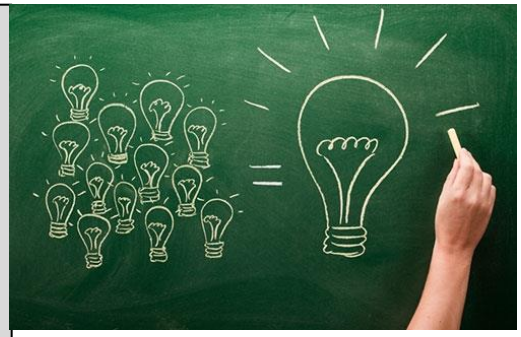
right now Pi Nerds can only memorize up to the π.

they can they like!

```
Nerd notSoNerdy(5);
Nerd superNerdy(100);

notSoNerdy.showOff();
superNerdy.showOff();
}
```

Copy Construction…
What's the big picture?

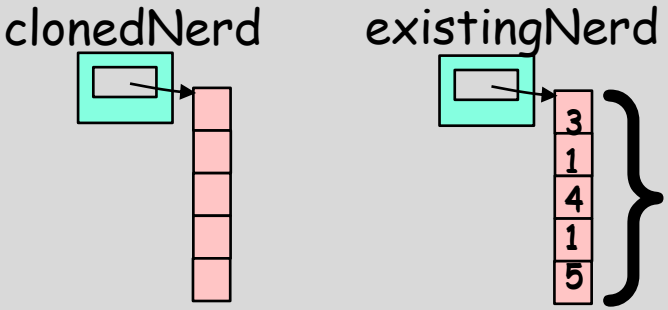
Copy construction is when we create (construct) a new object by copying the value of an existing object.



```
void cloneANerd()
{
   PiNerd existingNerd(4); // knows PI to 4 digits
   ...

   PiNerd clonedNerd = existingNerd;

   clonedNerd.showOff();  // prints 3.141
}
```

Create a new variable

By copying an existing variable



To clone more complex objects, we need to create a special function to do this, called a copy constructor.

Uses:

Copy constructors are required if you want to let users make a copy of more complex class variables.

Copy Construction

```
class Circ
{
public:
  Circ(float x, float y, float r)
  {
    m_x = x;   m_y = y;   m_rad = r;
  }

  Circ(const Circ& old)
  {
    m_x = old.m_x;
    m_y = old.m_y;
    m_rad = old.m_rad;
  }

  float GetArea() const;
private:
  float m_x, m_y, m_rad;
};
```

The parameter to our new constructor is another circle that we want to copy from!

That makes my head spin!

Then we just copy the member variables from the original object to the new object.

Creates a new circle...

existing circle!

```
{
  Circ a(1, 2, 3);

  Circ b(a);
}
```

Just as we can define a constructor that initializes a new Circ variable based on (x,y) and radius values…

We can also define a constructor that initializes a new Circ variable based on an existing Circ variable.

But wait! Circ variable b is accessing the private variables/functions of Circ variable a – isn't that violating C++ privacy rules?

b

```
class Circ
{
 ...
Circ( const Circ &old )
 {
 m_x = old.m_x;
```

a

```
class Circ
{
 ...
 Circ(float x, float y, float rad)
 {
   m_x = x;
   m_y = y;
   m_rad = rad;
 }
 ...
private:
  m_x 1  m_y 2  m_rad 3
```

```
main()

Ci    a(1,2,3);

Circ b(a);
...
}
```

That's not a problem. Every Circ variable is allowed to "touch" every other Circ variable's privates – "private" protects one class from another, *not* one variable from another (of the same class)!

So every CSNerd object can touch every other CSNerd object's privates.

But a CSNerd can't touch an EENerd's privates (for obvious reasons).

Copy Construction

```
class Cir
{
public:
  Circ(fl
  {
     m_x =
  }
  Circ(const Circ & old)
  {
    old.m_x = 10; // error 'cause of const
    m_x = old.m_x;
    m_y = old.m_y;
    m_rad = old.m_rad;
  }
  float GetArea()
  {
    return(3.14159*m_rad*m_rad);
  }
private:
  float m_x, m_y, m_rad;
};
```

This is c
won't
variable
you

This one's a bit more difficult to explain right now.

For now, just make sure you use an & here!

arameter to your
y constructor
uld be const!

The parameter to your copy constructor *must* be a reference!

The type of your parameter must be the same type as the class itself!

Copy Construction

```
class Circ
{
public:
  Circ(float x, float y, float r)
  {
    m_x = x; m_y = y; m_rad = r;
  }
  Circ(const Circ & old)
  {
    m_x = old.m_x;
    m_y = old.m_y;
    m_rad = old.m_rad;
  }
  float GetArea()
  {
    return(3.14159*m_rad*m_rad);
  }
private:
  float m_x, m_y, m_rad;
};
```

Oh, C++ also allows you to use a simpler syntax…

Instead of writing:

Circ b(a);

which is ugly…

You can write:

Circ b = a;

It does exactly the same thing! It defines a new variable b and then calls the copy constructor!

```
int main()
{
   Circ a(1,2,3);

   Circ b = a; // same!
}
```

Copy Construction

```
class Circ
{
public:
  Circ(float x, float y, float r)
  {
     m_x = x; m_y = y; m_rad = r;
  }
  Circ(const Circ & old)
  {
    m_x = old.m_x;
    m_y = old.m_y;
    m_rad = old.m_rad;
  }
  float GetArea()
  {
     return(3.14159*m_rad*m_rad);
  }
private:
  float m_x, m_y, m_rad;
};
```

The copy constructor is not just used when you initialize a new variable from an existing one:

Circ b(a);

It's used *any time* you make a new copy of an existing class variable.

Can anyone think of other times when a copy constructor would be used?

Copy Construction

temp

```
class Circ
{
 ...
 Circ(const Circ &old)
 {
 m_x = old.m_x;
 m_y = old.m_y;
 m_rad = old.m_rad;
 }
 ...
private:
  m_x  m_y  m_rad
}
```

We're calle valu

a

```
class Circ
{
 ...
 Circ(float x, float y, float rad)
 {
   m_x = x;
   m_y = y;
   m_rad = rad;
 }
```

```
void foo(Circ temp)
{
cout << "Area is: "
     << temp.GetArea();
}

int main()
{
Circ a(1,2,10);

foo(a);
}
```

Here's a simple program that passes a circle to a function…

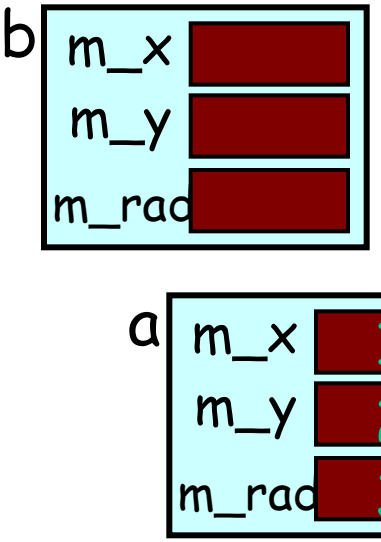
Any guesses if/when the copy constructor is called?

Now that our temp variable has been copy-constructed, it can be used normally by our foo function!

Copy C

But then why would I ever need to define my own copy constructor?

```
class Circ
{
public:
  Circ(float x, float y, float r)
  {
     m_x = x; m_y = y; m_rad = r;
  }
  Circ(const Circ& old)
  {
    m_x = old.m_x;
    m_y = old.m_y;
    m_rad = old.m_rad;
  }
  float GetArea()
  {
    return(3.14159*m_rad*m_rad);
  }
private:
  float m_x, m_y, m_rad;
};
```



It just copies all of the member variables from the old instance to the new instance…

This is called a "shallow copy."

```
int main()
{
  Circ a(1,2,3);

  Circ b(a);
}
```

Copy Construction

Ok – so why would we ever need to write our own Copy Constructor function?

After all, C++ shallow copies all of the member variables for us automatically if we don't write our own!

Well, we'll see very soon.

But first, let's go back to our PiNerd class…

The PiNerd Class

```
class PiNerd
{
public:
  PiNerd(int n) {
    m_n = n;
    m_pi = new int[n];
    for (int j=0;j<n;j++)
      m_pi[j] = getPiDigit(j);
  }

  ~PiNerd(){delete []m_pi;}

  void showOff()
  {
    for (int j=0;j<m_n;j++)
      cout << m_pi[j] << endl;
   }
private:
    int *m_pi, m_n;
};
```

When constructed, it uses new to dynamically allocate an array to hold the first N digits of π.

s you recall, every PiNerd orizes the first N digits of π.

Also recall that PiNerd uses new and delete to dynamically allocate memory for its array of N digits.

Let's see what happens when we use this class in a simple program.

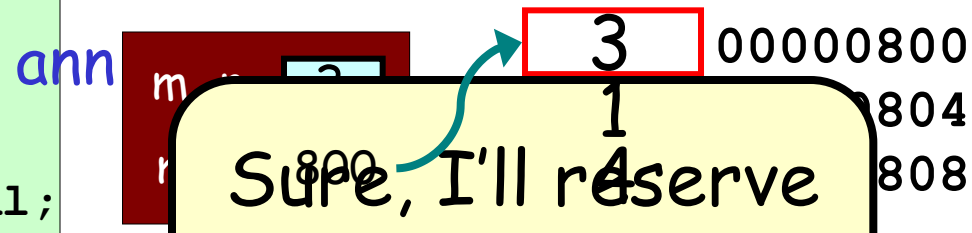
And when it is destructed, it uses delete [] to release this array.

Copy Construction

Operating system, can you reserve 12 bytes of memory for me?

```
class PiNerd
{
public:
  PiNerd(int
    m_n = n;
    m_pi = new int[n];
    for (int j=0;j<n;j++)
        m_pi[j] = getPiDigit(j);
}

  ~PiNerd(){delete []m_pi;}

  void showOff()
  {
    for (int j=0;j<m_n;j++)
      cout << m_pi[j] << endl;
   }
private:
    int *m_pi, m_n;
};
```

```
n(3);

  {
      PiNerd ben = ann;
      ...
  }

  ann.showOff();
}
```



Sure, I'll reserve 12 bytes for you at address 800.

struction

Now, watch what happens when we create our new ben variable and shallow copy ann's member variables into it…

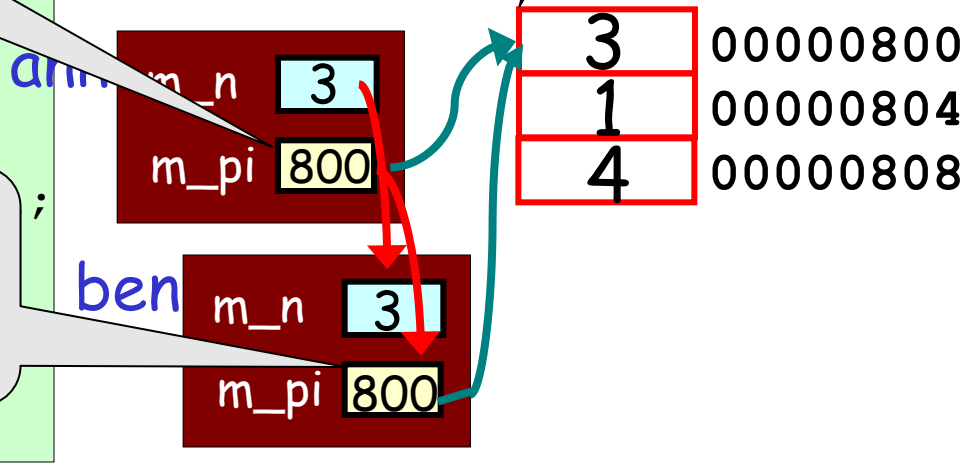
```
public:
   PiNerd(int n) {
     m_n = n;
     m_pi = new int[n];
     for (
       m_
   }
   ~PiNerd(){
   void showOff()
   {
     for (int j=0;j<m_n;j++)
       cou
    }
private:
    int *
};
```

```
int main()
{
   PiNerd ann(3)
   if (...)
   {
      PiNerd ben
      ...
   }
   ann.showOff()
}
```

Point to ann's original copy of the array!

Both ann's m_pi pointer…

And ben's m_pi pointer…



struction

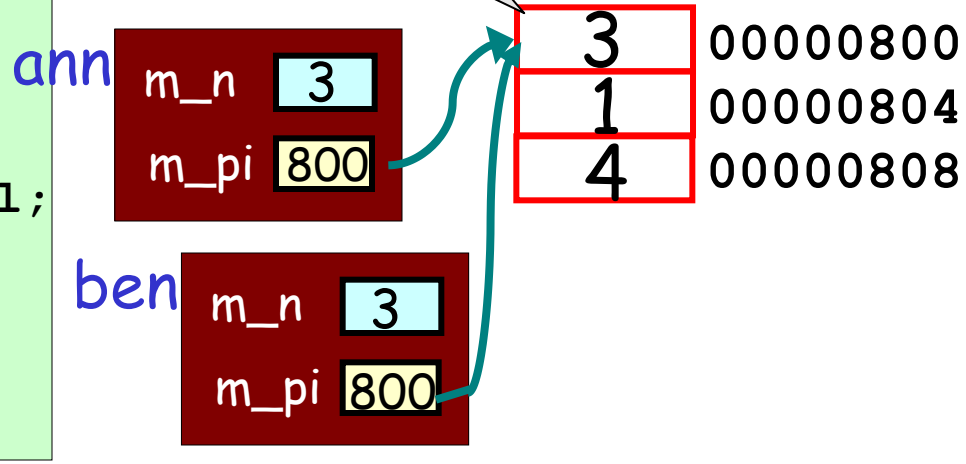
But that's a problem!

Because when ben is destructed...

```
class PiNerd
{
public:
  PiNerd(int n) {
    m_n = n;
    m_pi = new int[n
    for (int j=0;j<n
      m_pi[j] = getP
  }

  ~PiNerd(){delete []m_pi;}

  void showOff()
  {
    for (int j=0;j<m_n;j++)
      cout << m_pi[j] << endl;
   }
private:
    int *m_pi, m_n;
};
```

```
int main()
{
                    ) ;
                    h = ann;
                    called
  ann.            Off();
}
```

Operating system, I'm done with the memory at address 800.

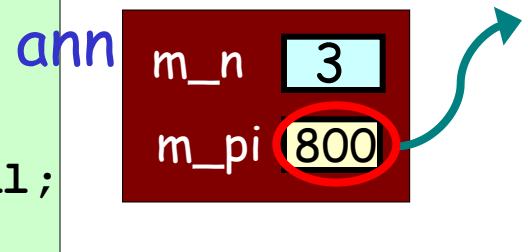


Copy Construction

```
class PiNerd
{
public:
   }

   ~PiNerd(){delete      _pi;}

   void showOff()
   {
     for (int j=0;j<m_n;j++)
       cout << m_pi[j] << endl;
    }
private:
    int *m_pi, m_n;
};
```

Because now, when we try to access ann's array, we get garbage!!!

```
int main()
{
   PiNerd ann(3);
   if (...)
   {
       PiNerd ben = ann;
       ...
   }// ben's d'tor called

   ann.showOff();
}
```



That's a big problem!

Copy Construction

...ral of the story?

And you make a shallow copy of a class instance...

BAD THINGS will happen when you destruct either copy...

Any time your class holds pointer member variables*...

```
class P
{
public:
  PiNerd(int n) {
    m_n = n;
    m_pi = new int[n];
    ...
    ...i;}
  void showOff()
  {
    for (int j=0;j<m_n;j++)
      cout << m_pi[j] << endl;
  }
private:
  int *m_pi, m_n;
};
```

```
int main()
{
  PiNerd ann(3);
  if (...)
  {
    PiNerd ben = ann;
    ...
  }// ben's d'tor called
  ann.showOff();
}
```

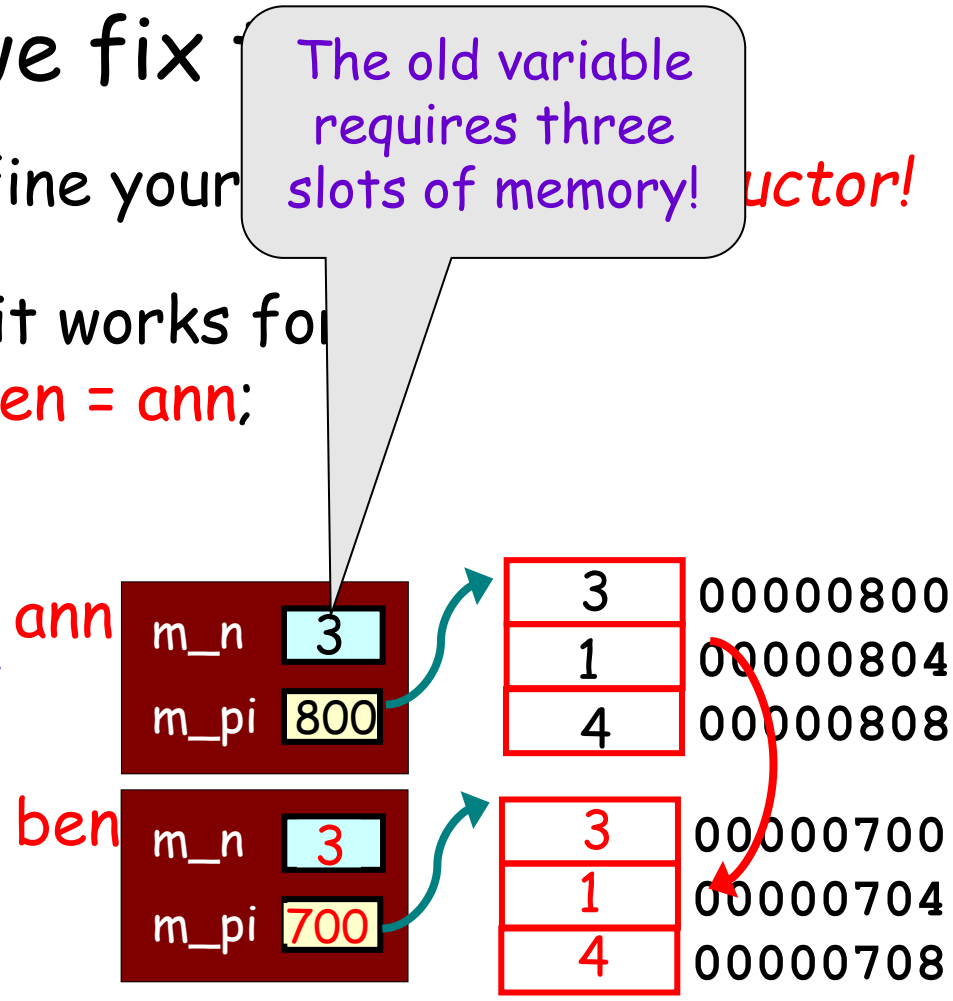
* or file objects (e.g., ifstream), network sockets, etc.

Copy Construction

So how do we fix

For such classes, you must define your uctor!

Here's how it works fo
PiNerd ben = ann;



1. Determine how much memory is allocated by the old variable.
2. Allocate the same amount of memory in the new variable.
3. Copy the contents of the old variable to the new variable.

The Copy Constructor

```
class PiNerd
{
public:
   PiNerd(int n) { ... }
   ~PiNerd(){ delete
   // copy con    uctor
   PiNerd(co    st PiNerd &src)
   {
     m_n = src.m_n;



   }
   void showOff() { ... }
private:
    int *m_pi, m_n;
};
```

```
      ...
   }
```

This means:
"The new instance must have the same number of array slots as the old instance."

First our copy constructor must determine how much memory is required by the new instance.

Let's see how to define our copy constructor!

The Copy Constructor

```
class PiNerd
{
public:
   PiNerd(int n) { ... }
   ~PiNerd(){ delete[]m_
   // copy constru
   PiNerd(const   Nerd &src)
   {
     m_n =  src.m_n;
     m_pi = new int[m_n];


   }
   void showOff() { ... }
private:
    int *m_pi, m_n;
};
```

```
   }

   ann.showOff();
}
```

This ensures that the new instance has its own array and doesn't share the old instance's array!

Next, our copy constructor must allocate its own copy of any dynamic memory!

Let's see how to define our copy constructor!

The Copy Constructor

```
class PiNerd
{
public:
   PiNerd(int n) { ... }
   ~PiNerd(){ delete[]m_pi; }

   // copy constructor
   PiNerd(const PiNerd &src)
   {
     m_n = src.m_n;
     m_pi = new int[m_n];
     for (int j=0;j<m_n;j++)
       m_pi[j] = src.m_pi[j];
   }

   void showOff() { ... }

private:
    int *m_pi, m_n;
};
```

```
int main()
{
  ann.showOff();
}
```

This ensures that the new instance has its own copy of all of the array data!

Finally, we have to manually copy over the contents of the original array to our new array.

Let's see how to define our copy constructor!

The Copy Constructor

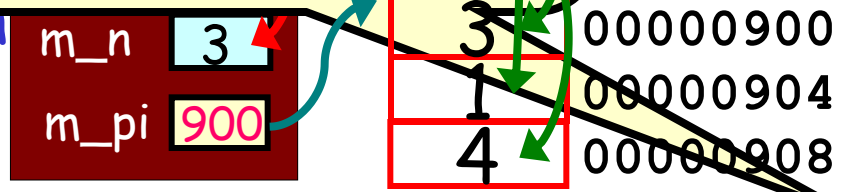
```
class PiNerd
{
public:
  PiNerd(int
  ~PiNerd()

  // copy co
  PiNerd(con
  {
    m_n = src.
    m_pi = new i
    for (int j=0;j<m_n;j++)
      m_pi[j] = src.m_pi[j];
  }

  void showOff() { ... }

private:
    int *m_pi, m_n;
};
```

```
int main()
{
                    = ann;
}
```

Operating system, the

Now I'll copy the values from the old array into my new array.

Sure, I'll reserve 12 bytes for you at address 900.



Let's watch our correct copy constructor work!

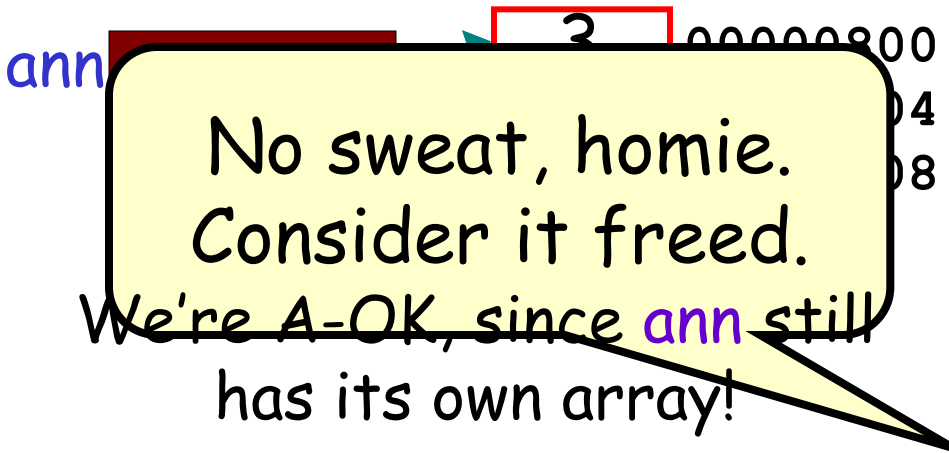
The

Operating System, can you free the memory at address 900 for me?

```
class PiNerd
{
public:
   PiNerd(int n) { ... }
→  ~PiNerd(){ delete[]m_pi; }

   // copy constructor
   PiNerd(const PiNerd &src)
   {
     m_n = src.m_n;
     m_pi = new int[m_n];
     for (int j=0;j<m_n;j++)
       m_pi[j] = src.m_pi[j];
   }

→  void showOff() { ... }

private:
    int *m_pi, m_n;
};
```

```
   if (...)
   {
       PiNerd ben = ann;
       ...
→  }// ben's d'tor called

→  ann.showOff();
}
```

3
1
4



No sweat, homie. Consider it freed. We're A-OK, since ann still has its own array!